Contracts và mục đích của từng contract

Legal-Ai-Compliance-Checker • BRD v1.1 • Data model v7.3.2 Đối chiếu workspace ngày 15/10/2026, bao gồm thay đổi chưa commit; dataset structured r2 / supabase-public-v1.

Legal-Ai-Compliance-Checker

Hướng dẫn tiếng Việt để đọc code, chia việc và hiểu dữ liệu đi qua từng module.

Contract trong tài liệu này là thỏa thuận dữ liệu giữa các module. Nó quy định gửi gì, nhận gì và kiểm tra gì. Nó khác với hợp đồng PDF/DOCX mà hệ thống đang đánh giá.

Ví dụ: Retrieval nhận checklist C05 và các đoạn hợp đồng; nó trả C06 là danh sách đoạn phù hợp với từng criterion. Compliance sử dụng dữ liệu đó để tạo C07.

Một contract không tự gọi DB, LLM hay OCR. File Pydantic định nghĩa cấu trúc; service/workflow mới thực hiện công việc. Một contract cũng không bắt buộc tương ứng với một bảng DB riêng.

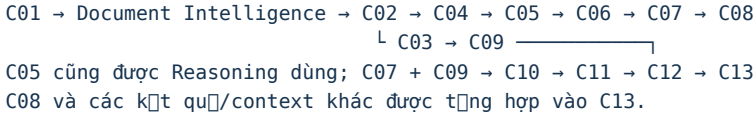
Phần	Nội dung
Nhìn toàn cảnh	Luồng hai nhánh và bảng tra cứu C01–C13 / X01–X02
Hiểu từng contract	Mục đích, producer/consumer, fields và hiện trạng
Hiểu bước 5–7	Ví dụ nối requirement, criterion, evidence và rủi ro
Dùng trong code	API contracts, DTO bàn giao và dữ liệu DB local

Cách đọc: bắt đầu ở sơ đồ luồng, tìm mã contract trong bảng tra cứu rồi đọc phần chi tiết. Các trường được liệt kê là trường tiêu biểu, không phải toàn bộ schema.

Đối chiếu: BRD v1.1; data_contracts_v7_3_2.md; workspace backend ngày 15/10/2026. Khi tài liệu thiết kế và model hiện tại khác nhau, phần “hiện trạng” mô tả đúng code hiện có.

Luồng dữ liệu: hai nhánh độc lập

Sơ đồ mô tả vai trò nghiệp vụ; các bước có interface chưa có nghĩa đã có implementation chạy thật.



C05 đi tới cả Retrieval và Reasoning. C06 không thay thế checklist/phiên bản. Contract RiskInput nhận C07 + C09; implementation risk hiện là skeleton. C08 phục vụ tổng hợp và report.

Citation VALID không đồng nghĩa hợp đồng COMPLIANT. C09 không sửa scope hoặc kết quả C08. Knowledge cung cấp tiêu chí/rules; X01 pin release, X02 truy vết provider calls thật.

Tra cứu nhanh tất cả contract

Mã C là dữ liệu nghiệp vụ; mã X là dữ liệu dùng chung cho phiên bản triển khai và audit.

Mã	Trả lời câu hỏi gì?	Module tạo module dùng
C01	Xác định đang đánh giá hợp đồng nào, file nguồn ở đâu và dùng cấu hình nào.	Intake/Backend Document Intelligence; workflow giữ thông tin này suốt lần đánh giá.
C02	Tóm tắt những thông tin thực sự đọc được từ hợp đồng để các bước sau xác định luật áp dụng.	Document Intelligence Scope, Temporal; Citation Validation cũng dùng profile làm ngữ cảnh.
C03	Ghi lại các văn bản/điều/khoản mà chính hợp đồng có nhắc tới.	Document Intelligence Citation Validation.
C04	Xác định nhóm yêu cầu nào có thể áp dụng cho hồ sơ, trước khi chọn phiên bản theo thời gian.	Scope Resolver Temporal Resolver.
C05	Nói rõ yêu cầu nào, phiên bản nào và mốc thời gian nào được dùng để đánh giá hồ sơ.	Temporal Resolver Retrieval và Compliance; cũng được giữ trong report/context.
C06	Liên kết criterion với những đoạn hợp đồng có thể dùng làm bằng chứng.	Retrieval Compliance Reasoning.
C07	Kết luận riêng cho một tiêu chí nhỏ, có giải thích, evidence, căn cứ và mức độ chắc chắn.	Compliance Reasoning Requirement Aggregator và Risk Engine.
C08	Gộp các C07 thuộc cùng một yêu cầu lớn thành một trạng thái requirement.	Requirement Aggregator Report/UI. RiskInput nhận C07/C09; risk mapper chưa triển khai.
C09	Đánh giá viện dẫn trong C03 có được nhận diện và phù hợp ở thời điểm/ngữ cảnh liên quan không.	Citation Validator Risk Engine và Report; nhánh độc lập với scope/compliance.
C10	Biến findings thành các rủi ro cụ thể theo rules và danh mục rủi ro.	Risk Engine Recommendation, Report/UI.
C11	Đề xuất hành động cho từng rủi ro, dựa vào template, evidence và căn cứ đã có.	Recommendation Human Review và Report/UI.
C12	Ghi ai đã duyệt/sửa/nhận xét kết quả nào và lý do thay đổi.	Reviewer / Review Service lịch sử review và quy trình report publishing.
C13	Gom toàn bộ kết quả để người dùng đọc, truy vết và xuất báo cáo.	Report Builder UI / Export; publishing cần quy trình review/quyền riêng.
X01	Ghi lại phiên bản schema, rules, knowledge và cấu hình model/prompt của một release.	Knowledge/Release các module; case pin cấu hình cần dùng.
X02	Truy vết một lần gọi model: dùng provider/model/prompt nào, chi phí, thời gian và kết quả.	Module thực sự gọi LLM audit/logging repository.

C01–C03: đọc và hiểu tài liệu nguồn

C01 — Hồ sơ ca đánh giá

Mục đích: Xác định đang đánh giá hợp đồng nào, file nguồn ở đâu và dùng cấu hình nào.

Ai gửi / ai nhận: Intake/Backend Document Intelligence; workflow giữ thông tin này suốt lần đánh giá.

Dữ liệu chính: ca_id, hop_dong_id, source, assessment_context, deployment, tenant_id, created_by.

Ví dụ: Hồ sơ CA_001 trỏ tới hợp đồng PDF, có mã băm của file và ngày đánh giá.

Điều cần nhớ: Source cần object_uri hoặc url_nguon và SHA256. Ca đánh giá phải giữ pin hợp đồng/deployment/knowledge nhất quán.

Model: AssessmentCase • **File:** shared/contracts/assessment.py

Hiện trạng: Đã có schema và interface Intake; chưa có quy trình upload/đăng ký hoàn chỉnh.

C02 — Profile hợp đồng

Mục đích: Tóm tắt những thông tin thực sự đọc được từ hợp đồng để các bước sau xác định luật áp dụng.

Ai gửi / ai nhận: Document Intelligence Scope, Temporal; Citation Validation cũng dùng profile làm ngữ cảnh.

Dữ liệu chính: profile, subjects, time_markers, facts, evidence; ví dụ loại hợp đồng, ngày ký, chủ thể, giá trị.

Ví dụ: Đọc được ngày ký 15/01/2026 thì ghi ngày, dữ kiện gốc và đoạn/trang chứng minh.

Điều cần nhớ: Không đọc được thì giữ null. Giá trị profile đã biết cần liên kết fact và evidence. Chưa biết một boolean không được tự đổi thành false.

Model: ContractProfile • **File:** shared/contracts/document_intelligence.py

Hiện trạng: Schema typed và kiểm tra provenance đã có; chưa có extractor/OCR thật.

C03 — Viện dẫn được trích từ hợp đồng

Mục đích: Ghi lại các văn bản/điều/khoản mà chính hợp đồng có nhắc tới.

Ai gửi / ai nhận: Document Intelligence Citation Validation.

Dữ liệu chính: vien_dan_id, hop_dong_id, raw_text, normalized, extraction, evidence.

Ví dụ: Một câu trong hợp đồng dẫn tên luật: lưu nguyên câu và vị trí xuất hiện để kiểm tra sau.

Điều cần nhớ: Trích được viện dẫn chưa có nghĩa viện dẫn đúng. C03 không được dùng để quyết định checklist luật áp dụng.

Model: CitationExtraction • **File:** shared/contracts/citation.py

Hiện trạng: Đã có schema; chưa có citation extractor thật.

C04–C06: chọn yêu cầu và tìm evidence

C04 — Kết quả xác định phạm vi

Mục đích: Xác định nhóm yêu cầu nào có thể áp dụng cho hồ sơ, trước khi chọn phiên bản theo thời gian.

Ai gửi / ai nhận: Scope Resolver Temporal Resolver.

Dữ liệu chính: yeu_cau_ca_id, requirement_id, nhom_phien_ban, scope_status, profile_snapshot_json, dieu_kien_snapshot_json, scope_reason.

Ví dụ: Một nhóm yêu cầu được đánh dấu MATCH, NO_MATCH hoặc UNKNOWN theo profile và knowledge.

Điều cần nhớ: Chưa đủ thông tin thì UNKNOWN. Scope chỉ dùng profile và knowledge, không dùng viện dẫn C03 để chọn luật.

Model: ScopeResolution • **File:** shared/contracts/scope.py

Hiện trạng: Đã có schema/interface và kiểm tra bàn giao; resolver thật chưa triển khai.

C05 — Yêu cầu và phiên bản áp dụng

Mục đích: Nói rõ yêu cầu nào, phiên bản nào và mốc thời gian nào được dùng để đánh giá hồ sơ.

Ai gửi / ai nhận: Temporal Resolver Retrieval và Compliance; cũng được giữ trong report/context.

Dữ liệu chính: yeu_cau_ca_id, requirement_id, requirement_version_id, scope_status, version_status, applicability_status, moc_ap_dung.

Ví dụ: C05 trỏ requirement REQ_A tới phiên bản REQ_A_V1, với mốc áp dụng đã xác định.

Điều cần nhớ: CONFIRMED cần MATCH + RESOLVED, có version và relevant date. Nếu chưa chọn được version, giữ UNKNOWN; không tự đoán.

Model: ApplicableRequirement • **File:** shared/contracts/temporal.py

Hiện trạng: Có schema C05 và validation context/checklist; Temporal Resolver và consumer nghiệp vụ bước 5 chưa triển khai.

C06 — Evidence được Retrieval tìm/chọn

Mục đích: Liên kết criterion với những đoạn hợp đồng có thể dùng làm bằng chứng.

Ai gửi / ai nhận: Retrieval Compliance Reasoning.

Dữ liệu chính: truy_xuat_id, yeu_cau_ca_id, tieu_chi_id, doan_id, scores, thu_hang, duoc_chon, giai_doan, versions, lan_chay_id.

Ví dụ: Criterion TC_A chọn đoạn D_03 ở hạng 1. Nội dung D_03 được lấy từ danh sách chunks.

Điều cần nhớ: C06 chủ yếu là tham chiếu, không chứa toàn bộ văn bản. C06 rỗng chưa chứng minh thiếu/vi phạm; cần xử lý uncertainty.

Model: RetrievalEvidence • **File:** shared/contracts/retrieval.py

Hiện trạng: Schema và validation đã chạy; search/index/reranker thật chưa triển khai.

C07–C08: đánh giá và tổng hợp

C07 — Kết quả đánh giá từng criterion

Mục đích: Kết luận riêng cho một tiêu chí nhỏ, có giải thích, evidence, căn cứ và mức độ chắc chắn.

Ai gửi / ai nhận: Compliance Reasoning Requirement Aggregator và Risk Engine.

Dữ liệu chính: ket_qua_id, yeu_cau_ca_id, tieu_chi_id, finding, giai_thich, gates, evidence, legal_basis_ids, uncertainty_reason, snapshot.

Ví dụ: TC_A có finding SATISFIED với quote hợp lệ; TC_B là UNKNOWN vì chưa đủ evidence.

Điều cần nhớ: Engine dùng C05 + C06 + chunks + knowledge + observations/facts. C07 là kết quả sau kiểm tra gates, không phải câu trả lời LLM chưa kiểm chứng.

Model: CriterionResult • **File:** shared/contracts/reasoning.py

Hiện trạng: Có schema C07; reasoner/grounding validator chỉ còn skeleton, phát sinh NotImplementedError.

C08 — Kết quả tổng hợp theo requirement

Mục đích: Gộp các C07 thuộc cùng một yêu cầu lớn thành một trạng thái requirement.

Ai gửi / ai nhận: Requirement Aggregator Report/UI. RiskInput nhận C07/C09; risk mapper chưa triển khai.

Dữ liệu chính: tuan_thu_id, yeu_cau_ca_id, status, số tiêu chí áp dụng/đã đánh giá/unknown, coverage_complete, snapshot, giai_thich.

Ví dụ: Một requirement có ba criterion; rule tổng hợp quyết định COMPLIANT, PARTIAL, NON_COMPLIANT hoặc INSUFFICIENT.

Điều cần nhớ: Nhóm theo yeu_cau_ca_id và checklist C05; version/coverage chưa rõ phải giữ uncertainty. C08 không được suy ra chỉ từ một đoạn hợp đồng.

Model: RequirementResult • **File:** shared/contracts/aggregation.py

Hiện trạng: Có schema C08 và rules trong knowledge; aggregator hiện chỉ là skeleton.

Kết quả	Ý nghĩa dễ hiểu
C07: SATISFIED	Có căn cứ cho thấy criterion được đáp ứng.
C07: MISSING	Đã xác minh thiếu nội dung bắt buộc; cần coverage/absence gates.
C07: VIOLATED	Có căn cứ cho thấy nội dung vi phạm criterion.
C07: UNKNOWN	Chưa đủ căn cứ để kết luận.
C08: COMPLIANT / PARTIAL	Requirement đáp ứng đầy đủ / một phần theo aggregation rule.
C08: NON_COMPLIANT / INSUFFICIENT	Không đáp ứng / chưa đủ dữ liệu để tổng hợp kết luận.

C09–C11: viện dẫn, rủi ro và hành động

C09 — Kết quả xác thực viện dẫn

Mục đích: Đánh giá viện dẫn trong C03 có được nhận diện và phù hợp ở thời điểm/ngữ cảnh liên quan không.

Ai gửi / ai nhận: Citation Validator Risk Engine và Report; nhánh độc lập với scope/compliance.

Dữ liệu chính: finding_id, vien_dan_id, normalized_van_ban_id, relevant_date, status, domain_status, reason_code, finding_verified, source_snapshot_json.

Ví dụ: Một viện dẫn là STALE; hợp đồng vẫn có thể COMPLIANT về nội dung, nhưng cần cảnh báo viện dẫn.

Điều cần nhớ: VALID, STALE, INVALID, UNKNOWN. Không đổi C05/C08 vì C09; trạng thái hiện tại của văn bản không tự thay thế đánh giá tại relevant date.

Model: CitationValidation • **File:** shared/contracts/citation.py

Hiện trạng: Có schema C09 và kiểm tra case/run; Citation Validator và Risk Engine thật chưa triển khai.

C10 — Kết quả rủi ro

Mục đích: Biến findings thành các rủi ro cụ thể theo rules và danh mục rủi ro.

Ai gửi / ai nhận: Risk Engine Recommendation, Report/UI.

Dữ liệu chính: ket_qua_rui_ro_id, risk_type, rui_ro_id, severity, source, quy_tac_id, evidence_ids, legal_basis_ids, snapshots.

Ví dụ: Một rủi ro trỏ tới C07 vi phạm; một rủi ro khác trỏ tới C09 cần xác minh.

Điều cần nhớ: Nguồn là đúng một criterion finding hoặc citation finding. Tách COMPLIANCE, CITATION, VERIFICATION; severity giữ theo catalog.

Model: RiskResult • **File:** shared/contracts/risk.py

Hiện trạng: Có schema C10; mappers/aggregator/service chỉ còn skeleton. Kết quả mới chưa được tạo.

C11 — Khuyến nghị xử lý

Mục đích: Đề xuất hành động cho từng rủi ro, dựa vào template, evidence và căn cứ đã có.

Ai gửi / ai nhận: Recommendation Human Review và Report/UI.

Dữ liệu chính: khuyen_nghi_ca_id, ket_qua_rui_ro_id, khuyen_nghi_id, noi_dung, loai_hanh_dong, uu_tien, evidence_ids, legal_basis_ids, template_snapshot_json.

Ví dụ: Nếu chưa đủ căn cứ, khuyến nghị yêu cầu xác minh thay vì tự đề xuất sửa điều khoản.

Điều cần nhớ: Không tự nâng severity. Giữ liên kết tới rủi ro/evidence/căn cứ và review_status=PENDING.

Model: Recommendation • **File:** shared/contracts/recommendation.py

Hiện trạng: Generator/validator/service chỉ là skeleton. Schema C11 hiện vẫn giới hạn model_version=TEMPLATE_ENGINE_NO_LLM và prompt_version=NOT_USED; metadata này không chứng minh engine đã chạy.

C12–C13: người review và báo cáo

C12 — Sự kiện người dùng review

Mục đích: Ghi ai đã duyệt/sửa/nhận xét kết quả nào và lý do thay đổi.

Ai gửi / ai nhận: Reviewer / Review Service lịch sử review và quy trình report publishing.

Dữ liệu chính: review_id, ca_id, target, actor_id, action, before, after, reason, reviewed_at, phien_ban_id.

Ví dụ: Reviewer yêu cầu bổ sung evidence cho một C07 UNKNOWN, ghi lại lý do và đối tượng review.

Điều cần nhớ: Actions: CONFIRM, OVERRIDE, REQUEST_MORE_EVIDENCE, REJECT, COMMENT. Lưu sự kiện append-only; giữ output AI gốc.

Model: HumanReview • **File:** shared/contracts/review.py

Hiện trạng: Schema/service/repository interfaces đã có; review và phân quyền thật chưa triển khai.

C13 — Báo cáo đánh giá

Mục đích: Gom toàn bộ kết quả để người dùng đọc, truy vết và xuất báo cáo.

Ai gửi / ai nhận: Report Builder UI / Export; publishing cần quy trình review/quyền riêng.

Dữ liệu chính: bao_cao_id, ca_id, phien_ban_bao_cao, report_json, report_status, review_status, report_uri.

Ví dụ: Report có profile, checklist, C07/C08, C03/C09, các nhóm rủi ro và khuyến nghị riêng.

Điều cần nhớ: Tách compliance/citation/verification. Builder hiện chỉ tạo DRAFT/PENDING; schema có PUBLISHED nhưng chưa có luồng publish.

Model: FinalReport • **File:** shared/contracts/report.py

Hiện trạng: Helper ghép report JSON nháp được giữ; caller phải truyền risks=C10 và recommendations=C11 đã hợp nhất. Builder không tự tính risks/actions; chưa persist/export/publish. Nội dung review C12 chưa có field riêng trong ReportContent hiện tại.

Có model FinalReport với trạng thái PUBLISHED không có nghĩa hệ thống hiện tự publish được. Builder hiện chỉ tạo JSON nháp DRAFT/PENDING. Human review, phân quyền, export và publication cần implementations riêng.

X01–X02 và Context: truy vết lần chạy

X01 — Snapshot triển khai và knowledge

Mục đích: Ghi lại phiên bản schema, rules, knowledge và cấu hình model/prompt của một release.

Ai gửi / ai nhận: Knowledge/Release các module; case pin cấu hình cần dùng.

Dữ liệu chính: phien_ban_id, schema_version, legal_snapshot_sha256, contract_snapshot_sha256, rule_version, model_reasoning, model_embedding, prompt_version.

Ví dụ: Một case tiếp tục dùng knowledge/rules đã pin dù hệ thống có release mới.

Điều cần nhớ: Version schema, version rules và version model là các khái niệm khác nhau. Không âm thầm đổi pin của case đã lưu.

Model: DeploymentSnapshot • **File:** shared/contracts/deployment.py

Hiện trạng: Schema/interface đã có; runtime pin bằng Context/DeploymentPin; chưa có quy trình publish release hoàn chỉnh.

X02 — Nhật ký gọi LLM

Mục đích: Truy vết một lần gọi model: dùng provider/model/prompt nào, chi phí, thời gian và kết quả.

Ai gửi / ai nhận: Module thực sự gọi LLM audit/logging repository.

Dữ liệu chính: lan_goi_id, ca_id, module, provider, model_version, prompt_version, input/output_sha256, tokens, latency_ms, chi_phi, trang_thai, trace_id.

Ví dụ: Một lần gọi LLM lỗi được ghi ERROR, mã lỗi và số lần thử; có trace nối với run.

Điều cần nhớ: Chỉ ghi khi có provider call thật. Snapshot/audit của run PostgreSQL không phải log một lần gọi LLM; chỉ tạo X02 khi thực sự gọi provider.

Model: LlmAudit • **File:** shared/contracts/audit.py

Hiện trạng: Có PostgresLlmAuditSink và audit_llm_call dùng chung; Services.llm_audit inject sink vào module. Client/provider/prompt do từng module tự cấu hình. Sink ghi metadata/hash vào runtime.nhat_ky_goi_llm, không gọi LLM hoặc lưu API key/raw prompt. Token/chi phí chưa biết giữ null.

Context là “nhãn” đi cùng dữ liệu

Nhóm trường	Dùng để làm gì?
ca_id; run_id; trace_id	Phân biệt hồ sơ, lần chạy và chuỗi log xuyên module.
phien_ban_id; rule_version; legal_knowledge_version	Ghi cấu hình triển khai và phiên bản rules/knowledge được pin.
knowledge_snapshot_sha256	Kiểm tra đúng nội dung knowledge snapshot bất biến trong PostgreSQL.
contract_version = 1.0	Phiên bản interface trong Context; khác data-model schema v7.3.2.

X01 có legal_snapshot_sha256 và contract_snapshot_sha256; Context dùng knowledge_snapshot_sha256. Đây là các tên trường khác nhau, không phải một cơ chế release đã tự đồng bộ. Các model C không đều tự chứa toàn bộ Context; gói request/handoff sẽ mang context cần thiết.

Ví dụ dễ hiểu: từ C05 đến C11

Ví dụ giả lập về cấu trúc dữ liệu; không phải kết luận cho một hợp đồng thật hoặc đáp án pháp lý đã xác thực.

1. C05: chọn requirement

YCCA_A là ID của yêu cầu trong hồ sơ. Nó trỏ tới REQ_A và phiên bản REQ_A_V1. Knowledge của phiên bản này có các criterion TC_1, TC_2, TC_3.

2. C06: chọn đoạn cho từng criterion

Retrieval trả các liên kết YCCA_A + TC_1 + D_01, YCCA_A + TC_2 + D_02... Văn bản của D_01/D_02 nằm trong chunks.

3. C07: đánh giá riêng từng criterion

Implementation bước 5 cần nhận C05, C06, chunks, knowledge và observations/facts; kiểm tra quotes, nguồn, scope, version và gates trước khi phát hành finding. Hiện entrypoint là skeleton.

4. C08: tổng hợp requirement

Các C07 được nhóm theo yeu_cau_ca_id = YCCA_A. Aggregator đọc rule của requirement để ra một C08; không nhóm chỉ theo chunk.

5. C10: xử lý hai nguồn độc lập

Compliance risk đọc C07. Citation risk đọc C09. Nếu có lỗi viện dẫn nhưng các criterion đều SATISFIED, C08 không tự đổi thành NON_COMPLIANT.

6. C11: khuyến nghị theo rủi ro

Template được chọn theo risk/rule/link. Thiếu căn cứ thì đề nghị xác minh. Kết quả vẫn chờ human review.

Phân biệt ba ID: requirement_id là yêu cầu chung; requirement_version_id là phiên bản của yêu cầu; yeu_cau_ca_id là yêu cầu đó trong một hồ sơ cụ thể. C06/C07/C08 liên kết qua yeu_cau_ca_id; criterion được xác định bằng tieu_chi_id.

Các bước trong ví dụ là luồng mục tiêu, chưa chạy bằng các stub hiện tại. Trong code hiện tại chưa có provider LLM. ObservationProvider.observe là interface đầu vào duy nhất được workflow dùng, trả các Observation. Gates cần kiểm tra thật, không tin flags do LLM tự khai báo; proposed finding chưa phải C07 đã xác minh.

Contracts nội bộ bước 5–7

Mã C07/C08… mô tả từng loại kết quả. Các request DTO đóng gói dữ liệu cùng Context để gọi hàm Python; không có HTTP API riêng cho từng module.

Model / contract_name	Nhận dữ liệu gì?	Bàn giao / hiện trạng
ReasoningRequest / CriterionReasoningInput	C05 + C06 + chunks + observations, không có C09	Input hàm reasoning; dự kiến C07
aggregate(Context, knowledge, criteria, requirements)	Context + knowledge + C07 + C05	Hàm tổng hợp requirement; dự kiến list C08
RiskRequest / RiskInput	C05 + C07 + C09	Input risk; dự kiến C10
RecommendationRequest / RecommendationInput	C05 + C10	Input recommendation; dự kiến C11
PipelineRequest / Steps57Input	C05 + C06 + chunks + observations + C09	Input workflow nội bộ 5–7; implementation còn skeleton
PipelineResponse / Steps57Result	Context + produced_at + C07/C08/C10/C11 + excluded_requirements	GET /api/v1/cases/{ca_id}/runs/{run_id} đọc run đã lưu; 404 nếu không có

Các model payload hỗ trợ

ReasoningPayload chứa dữ liệu đánh giá criterion; AggregationPayload chứa C05/C07; RiskPayload thêm C09; RecommendationPayload chứa C05/C10. PipelinePayload gom input bước 5–7 và từ chối duplicate IDs. PipelineResult giữ các list output của pipeline.

Các POST chạy riêng bước 5–7 đã xóa. GET run đọc snapshot Steps57Result đã lưu; GET knowledge đọc metadata snapshot PostgreSQL. Workflow/services bước 5–7 phát sinh NotImplementedError cho đến khi nhóm C triển khai; API khởi chạy assessment toàn bộ chưa có implementation.

File model: shared/contracts/reasoning.py, modules.py, pipeline.py. Thiết kế HTTP: docs/openapi.design.json; schema runtime xem tại /openapi.json của backend đang chạy.

DTO bàn giao khi phát triển song song bằng DB

Các model nằm trong shared/contracts/handoffs.py, tái sử dụng C01–C13. Module query rows của đúng case/run/hash, map sang DTO rồi kiểm tra contract. Không còn thư mục examples/parallel hoặc file JSON fixture cố định để chạy module.

Model / checkpoint	Dữ liệu và mục đích
DocumentInput	Context + C01; input cho người làm Document
DocumentIntelligenceResult	C02 + C03 + chunks; output của Document Intelligence
ScopeInput	Context + C02; không có field citations
TemporalInput	ScopeInput + C04; input phân giải C05
CitationInput	Context + C02 + C03 + chunks cho nhánh citation
LegalOutput	C04 + C05 + C09; các nhánh được xử lý độc lập
RetrievalInput	Context + contract ID + C05 + chunks
list[RetrievalEvidence]	C06: liên kết criterion với đoạn evidence
ReasoningRequest	Input bước 5: checklist/evidence/chunks/observations
ComplianceOutput	Context + C07/C08 và risks/actions từ criterion; schema từ chối citation-sourced risks
IntegrationInput	Context + case + document + legal + compliance để ghép report
PipelineResponse	Checkpoint C07–C11 dự kiến; có thể đọc từ run đã lưu
FinalReport	C13 draft từ các outputs được cung cấp

HandoffSet gom các checkpoint trên để xuất module_contracts.schema.json. Nó mô tả cấu trúc bàn giao, không chạy pipeline hoặc sinh fixtures.

Bộ DEV_VALIDATION có 240 DEV + 60 VALIDATION và 32 ca regression riêng trong QA. Runtime chứa rows tham chiếu để module có input khi upstream chưa hoàn thành. qa.case_manifest giữ split/version/hash; qa.artifacts giữ INPUT, EXPECTED hoặc PROPOSAL theo case và contract. qa.controlled_assertions giữ điều kiện mong đợi của ca kiểm thử.

Input mẫu dùng để phát triển; EXPECTED dùng để đối chiếu sau khi chạy implementation của mình, PROPOSAL giữ đề xuất chưa xác nhận. Không dùng đáp án tham chiếu làm output thật hoặc báo legal/LLM accuracy từ việc replay đáp án. MASTER có nhãn TEST dành cho evaluator; TEST_INPUTS chỉ chứa 60 PDF và intake, không chứa đáp án.

Nhóm A dùng bytes PDF từ public URL trong tep_nguon.object_uri, kiểm tra source SHA-256; profile/chunks/C03/DocumentStructure là mục tiêu tham chiếu. Nhóm B/C có thể đọc output upstream mẫu từ DB để làm song song. Khi integration, thay chúng bằng output thật, giữ case/run/provenance nhất quán.

C13 helper nhận IntegrationInput cùng hai tham số bắt buộc risks=merged_c10 và recommendations=merged_c11. Nhóm D cung cấp C10/C11 của đúng run, kể cả nhánh citation; report không chạy lại bước 6–7.

Các kiểu hỗ trợ và nơi tìm schema

Kiểu dữ liệu	Mục đích
Model	Pydantic base dùng chung: từ chối trường lạ và chuẩn hóa chuỗi thông thường.
SourceDocument; AssessmentTiming	Tham chiếu bytes/file nguồn, SHA256; ngày đánh giá và mốc thời gian.
DeploymentPin; Context	Giữ cấu hình case và nhận diện run/trace/version/checksum.
Chunk; Evidence; ExternalEvidence	Văn bản đoạn; quote + offsets; tài liệu evidence ngoài với mã băm và trạng thái verified.
ExtractedFact; ProfileEvidence	Dữ kiện trích xuất, phương pháp/trạng thái và liên kết tới bằng chứng.
ProfileFields; ProfileSubject; ProfileTimeMarker	Fields profile có thể null; chủ thể; mốc thời gian có khả năng dùng phân giải.
CitationNormalization; CitationExtractionMetadata; CitationPosition	Tên/số hiệu/điều khoản chuẩn hóa; trạng thái/confidence trích xuất; đoạn và khoảng trang.
Gates; Observation	Các điều kiện xác minh; nhận xét đầu vào cho criterion. LLM không được tự nhận quyền xác minh gates.
RiskSource; ReviewTarget	Trỏ đúng một nguồn risk; xác định đối tượng mà reviewer đang thao tác.
ReportContent; ReportCompliance; ReportCitations; ReportRisks	Các section typed của C13; giữ compliance và citation tách biệt, chia nhóm rủi ro.

Dữ liệu kiểm thử: workbook DEV_VALIDATION và các references được tải bằng scripts.dataset, seed bằng scripts.local_db. Tests phần mềm dùng DB test riêng và dữ liệu nhỏ; không phụ thuộc corpus private. Kiểm thử chất lượng OCR/search/LLM cần implementation thật và targets được kiểm tra độc lập.

Nguồn đối chiếu trong repo:

- BRD v1.1
- Contracts nghiệp vụ gốc
- Contracts typed trong code
- OpenAPI thiết kế UI và module schemas
- Tải dataset, DB local và JOIN theo nhóm
- Audit LLM dùng chung và schema X02
- Trạng thái backend và skeleton

Các trường trong hướng dẫn là giải thích rút gọn. Khi viết payload thật, dùng Pydantic model/OpenAPI/JSON Schema làm nguồn chuẩn. Phần mô tả bước xử lý là trách nhiệm dự kiến; các stub không phát hành kết luận. References của dataset chưa phải expert legal gold.